Computer Science & Engineering Department

# Arhitectura Sistemelor de Calcul

Curs 2
cs.pub.ro
cs.curs.pub.ro

University Politehnica of Bucharest



# In cursul anterior

- Notare
- Sumarul Cursului
- Bibliografie
- Educatia Inginereasca
- De ce Arhitectura Sistemelor de Calcul?
- Idei Importante in Arhitectura Sistemelor de Calcul

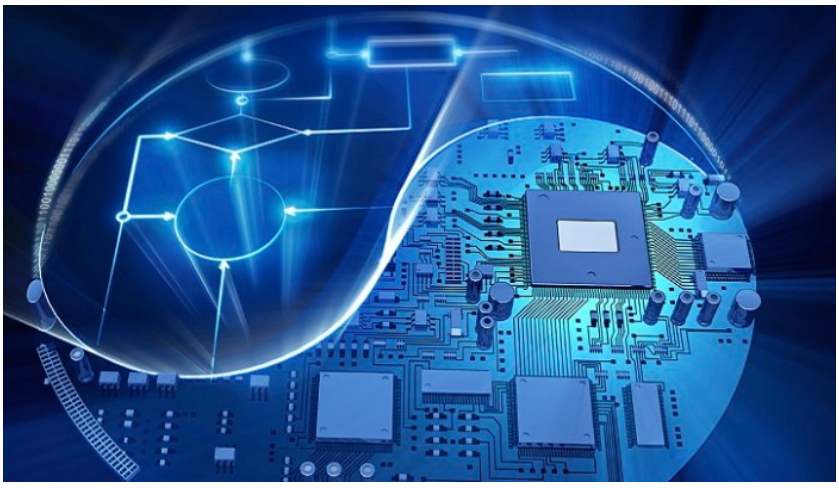




# Cuprins

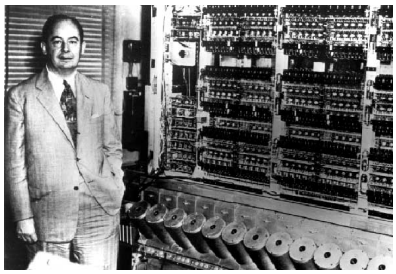
- Componentele Sistemelor de Calcul
- Nivelul Structural de Descriere al Sistemelor Numerice – PMS (Processor Memory Switches)
- Exemple de descrieri PMS
- Structuri multiprocesor organizate pe o magistrala comuna (SBC)





# Componentele Sistemelor de Calcul

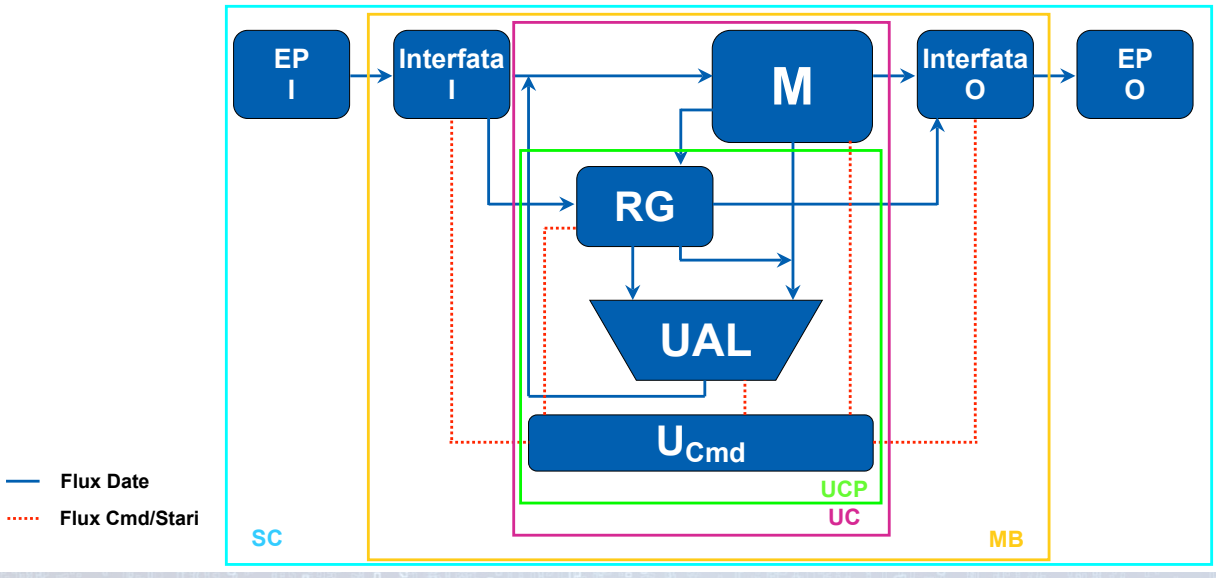
# Masina von Neumann



- John von Neumann (1903 – 1957)
- Structura cu acces secvential
  - CPU
  - Memorie (Instructiuni & Date)
  - Ex: EDVAC (1945) – binar
- *“There's no sense in being precise when you don't even know what you're talking about.”*
- *“In mathematics you don't understand things. You just get used to them.”*
- *“It would appear that we have reached the limits of what it is possible to achieve with computer technology, although one should be careful with such statements, as they tend to sound pretty silly in 5 years.” (Said in 1949)*

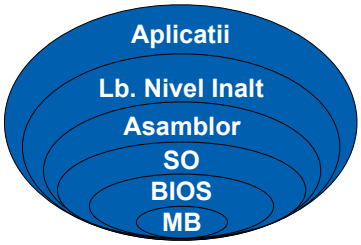


# Arhitectura masinii von Neumann



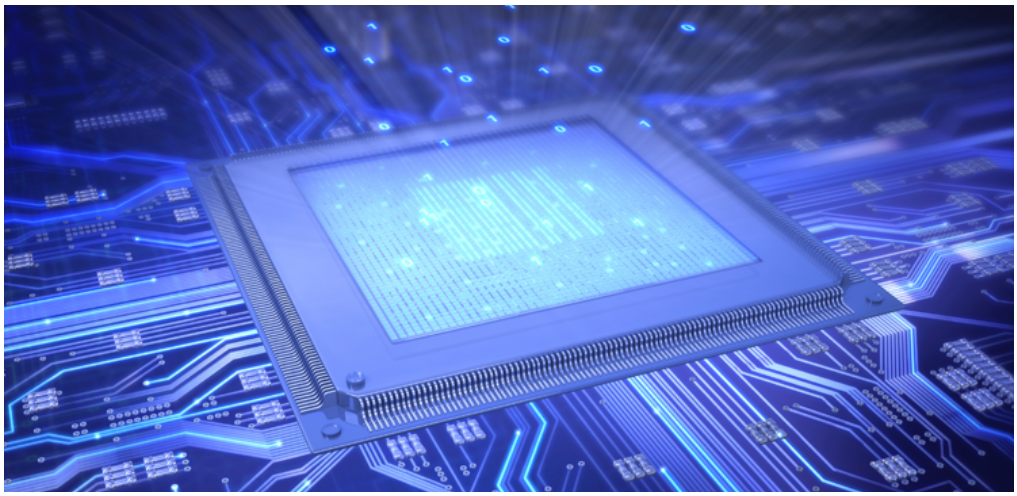
# Componentele Arhitecturii

- UCP = {RG, UAL, $U_{Cmd}$} = Unitatea Centrala de Prelucrare
- UC = {UCP, M} = Unitatea Centrala
- MB = {UC, I/E} = Masina de Baza
- SC = {MB, EP, Software de Baza} = Sistem de Calcul




Software de Baza





# Directive PMS

# Nivelul Structural de Descriere al Sistemelor Numerice

- PMS (Processor Memory Switches)
- La nivel structural sistemele numerice (SN) sunt specificate si analizate considerand urmatoarele elementele primare:
  - PC = procesorul central
  - M = memoria
  - S = comutatorul
  - L = linia de legatura
  - K = unitatea de comanda
  - PI/E = procesorul de intrare/iesire sau interfata
  - D = operatorul de date
  - T = terminalul





# Primitivele PMS

- Fiecare primitiva
  - Este caracterizata printr-un atribut:
    - $a_i$ = atributul asociat primitivei
- Fiecare atribut
  - Are o anumita valoare:
    - $v_i$ = valoarea corespunzatoare atributului
  - Astfel un sistem de calcul devine:
    - SN ($a_1$:$v_1$; $a_2$:$v_2$; … $a_n$:$v_n$)





# Memoria – M

- Rolul memoriei:
  - Pastreaza informatia
  - Actualizeaza informatia prin operatii de citire/scriere
  - Necesita un sistem de adresare cu o corespondenta liniara intre adresa si continut
- Atribute:
  - Functie: primara | auxiliara
  - Tehnologie: bipolara | MOS | statica | dinamica
  - Operatii: citire | citire/scriere
  - Mod acces: aleator | secvential | FIFO | LIFO | asociativ
  - Lungime cuvant: 8+1 | 16+2 | 32+4 | 64+8
  - Capacitatea: 4Mb | 8Mb | 16Mb | …| 1Gb | 2Gb | 4Gb | 8Gb | 16Gb | …
  - Ciclu de lucru: 4ms | … | 400ns | 200ns | 100ns | 50ns | …



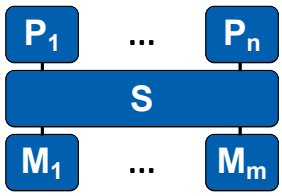
# Procesorul Central – $P_c$

- Rolul PC:
  - Citeste, interpreteaza si executa instructiuni masina
  - In acest proces are loc generarea comenzilor spre toate resursele sistemului + citirea starilor acestora
- Atribute:
  - Functie: universal | specializat (de semnale, etc)
  - Implementarea: conventionala | μprogramata | μprogramata specializata
  - Formatul instructiunilor: fix | variabil
  - Lungimea instructiunilor: 8 | 16 | 32 | 64 | 128 biti
  - Ciclu instructiune: ciclu fix | ciclu varibil (S cicli masina)
  - Tehnologia: LSI | VLSI | ULSI |…

# Switch/Comutatorul – S

- Rolul S:
  - Asigura conexiuni intre componentele sistemului
  - Poate evolua
    - De la structuri foarte simple = buffer 3 state de acces pe magistrala
    - La structuri complexe cu unitati de comanda proprii = comutatoare cu discipline de servire a cererilor de stabilire a legaturilor
- Atribute:
  - Structura: ierarhica | neierarhica
  - Tip: simplex | semiduplex | duplex (e vorba de sensuri)
  - Numar de legaturi realizate:
    - m intrari | n iesiri din S
    - n intrari | m iesiri din S
  - Concurenta: c

**Maxim de paralelism: min (n, m)**






# Legatura – L

- Rolul L:
  - Asigura legatura fizica intre diverse componente ale sistemului
  - Nu prelucreaza informatia ci doar asigura traseul transferului spatial al datelor
  - E fie o magistrala, fie o interfata seriala/paralela
- Atribute:
  - Functia: legatura seriala | legatura paralela | legatura de tip magistrala
  - Lungimea cuvantului: 5 | 6 | 7 | 8 | 16 | 32 | 64 + comanda
  - Mod de dialog: sincron | asincron | cu Q&A: cu/fara interblocare sau cu interblocare completa | fara Ack
  - Mod de control al accesului: inlantuire seriala (token) | interogare | cereri independente
  - Standard: paralel (SCSI) | serial (RS232; 485; 482) | magistrala (MultiBus; MCI(IBM); HPIBus; etc.)

# Unitatea de Comanda – K

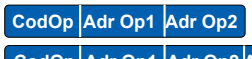
- Rolul K:
  - Componenta care exprima controlul in diverse subansamble ale sistemului
  - Are o functie de comanda a resurselor asociate unitatilor functionale
    - Nu citeste & interpreteaza instructiuni aritmetice logice, ci doar **comanda** & **control**
- Atribute:
  - Functia: comanda resursa I
  - Implementare: conventionala | μprogramata
  - Numar de stari: n
    - Daca are 2 stari este un bistabil
    - Daca are 64 de stari este un automat complex

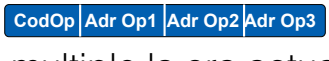


# Unitatea de Comanda – K

- Citireste, interpreteaza si executa instructiuni masina de **comanda si control**
- Legata de:
  - Setul de instructiuni
    - Intructiuni Universale; Ciclu variabil
    - RISC cu ciclu fix
    - Complex CISC – emularea unei masini intermediare
- Codificarea instructiunilor
  - 0 adrese 

  - 1 adresa 

  - 2 adrese 

  - 3 adrese 

- Moduri de adresare multiple la ora actuala

# Procesorul de I/E – $P_{I/E}$

- Rolul $P_{I/E}$:
  - Poate fi incorporat in PC-ul in care are functie de I/E sau poate fi privit independent
  - Daca este o primitiva independenta are urmatoarele atribute
- Atribute:
  - Functie: DMA | canal I/E | procesor specializat I/E
  - Tip implementare: conventional | μprogramat | (co)procesor specializat I/E
  - Rata de transfer: Mb/s | Gb/s | Tb/s
  - Mod de transfer: cum rezolva conflictul de acces la UC
    - Prin furt de ciclu
    - Prin rafala
  - Lungimea cuvantului: 8 | 16 biti (in general caractere)





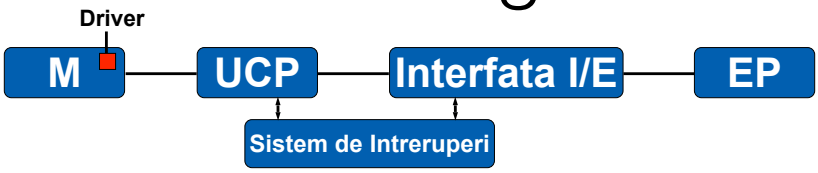
# Subsistemul de I/E

- Asigura transferul datelor intre UC si mediul extern
- Interfetele I/E asigura
  - O adaptare electrica intre EP si UCP
  - Sincronizarea temporala intre EP si UCP printr-un cuvant de stare
    - Diferenta este de cateva ordine de marime
  - Transfer de date intre EP si UC:
    - Transfer programat
    - Acces direct la memorie
    - Canal de intrare/iesire
    - Calculator de intrare/iesire (Front End Computer/Processor - FEP)





# Transfer Programat

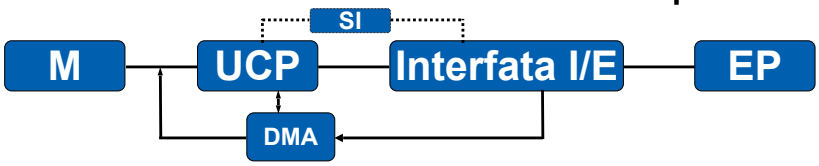



- UCP-ul se ocupa de fiecare caracter in parte
- Driver-ul se ocupa de transferul datelor I/O
- Sincronizarea se poate face
  - Prin citirea ciclica a starii (busy waiting)
    - UCP asteapta dupa EP si ajunge sa lucreze la viteza EP-ului
    - Driver-ul citeste si intrerupe cuvintele de date/stare/comanda
  - Prin intreruperi
    - Se dau ordine EP si se continua prelucrarile
    - Cand se termina se genereaza o intrerupere
    - UCP-ul nu se adapteaza la viteza EP-ului



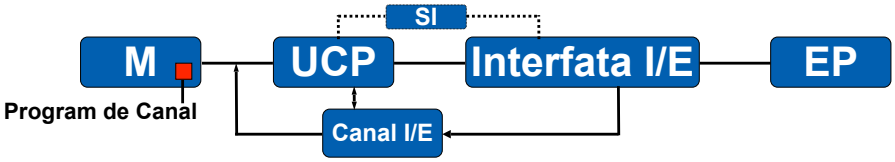


# Acces Direct la Memorie prin DMA




- Asigura transferul de date intre EP si M fara interventia UCP
- Dialogul intre UCP si DMA are loc doar la initiere & incheiere
- Dialogul se face prin transfer programat (Intreruperi)
- Initiere: UCP → DMA: adrese date, sensul (R/W), adresa EP
- La orice eroare DMA cheama UCP-ul
- UCP-ul nu poate initia un alt transfer pana nu s-a incheiat cel curent
- Arbitrarea conflictului de acces la M al UCP/DMA se face prin
  - Furt de ciclu
  - Rafala

# Canal de I/E




- Canalul I/E e un procesor specializat ce asigura transferul intre EP si M fara interventia UCP
- Dialogul UCP – EP se face printr-un program de canal (din M – se da doar adresa de inceput a programului)
- UCP poate inlantui mai multe programe de canal = mai multe transferuri de blocuri de date (mai bine ca la DMA)
- Canalul aduce informatia necesara transferului in registrele proprii prin DMA (acces direct)
- Canalul reincearca transferul in caz de eroare



# Calculator de I/E




- In cazul calculatorului de I/E, SC poate avea toate facilitatile de mai sus:
  - DMA
  - Canal I/E
  - Transfer programat
- Mecanism util pentru a nu irosi timpul unui procesor / calculator puternic
- Ofera datele rapid si eficient pentru prelucrare
- Este folosit des in sistemele multiprocesor

# Operatorul de Date – D/UAL

- Rolul D:
  - “Produce” unitati de informatie cu semnificatie noua
  - Efectueaza operatii aritmetice & logice + prelucrari primare de compactare, expandare si asociere asupra datelor
  - Este unitate de prelucrare: in virgula mobila si zecimala
  - Sunt module optionale, de sine statatoare, independente de PC
- Atribute:
  - Functie: prelucrare in virgula mobila | zecimala(BCD) | vectoriala | matriceala | cu liste
  - Tip implementare: conventionala | procesor specializat (coprocesor matematic)
  - Structuri de date asupra carora opereaza: scalari | vectori | matrice | liste
  - Operatii: + | - | * | / | cautare atomica | memorare/extragere atomica

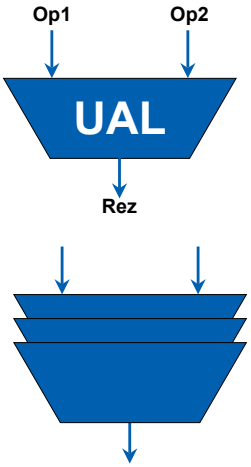


# Evolutia Operatorului de Date D/UAL

- UAL: in virgula fixa si virgula mobila
- Formatul general al instructiunilor




- Evolutia spre SIMD (masini vectoriale)
  - UAL multiple
  - Fiecare element are o prelucrare dedicata
  - Rezultatele sunt predate etajului adiacent
- Op UAL: +, -, *, /, ^, %, Shiftari, etc
- In timp UAL-ul s-a extins catre coprocesorul matematic



# Terminalul – T

- Rolul T:
  - Asigura conversia din punct de vedere fizic a datelor
    - Adaptare electrica
    - Asigura sincronizarea intre terminal si UC de prelucrare sau PI/E care se ocupa de acel terminal
  - Terminalul este format fie
    - Dintr-o singura componenta
    - Din 3 componente
      - Terminal
      - Unitate de comanda a T: interpreteaza comenzi/stari de la UC
      - Switch T




- Atribute:
  - Functie: cupleaza terminalul i
  - Tip cuplare: seriala | paralela
  - Caracteristici: viteza de transfer | capacitate | pagini/min

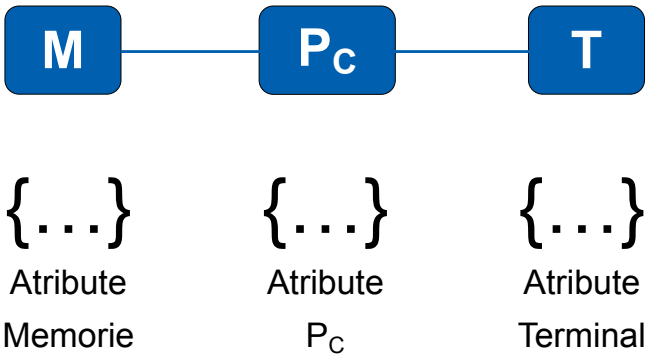





# Exemple de Descrieri PMS

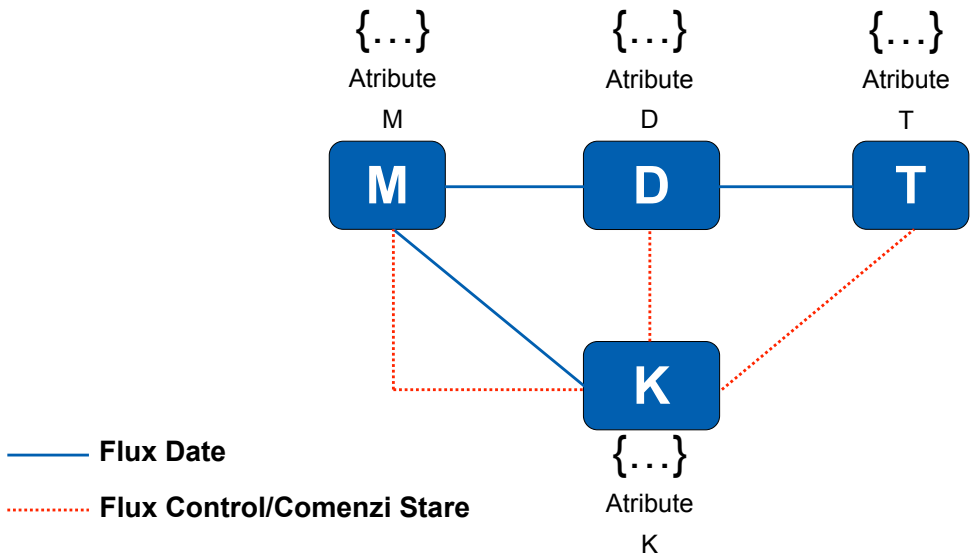
# Exemplu de Folosire ale Primitivelor PMS

- Sa incercam o structura von Neumann

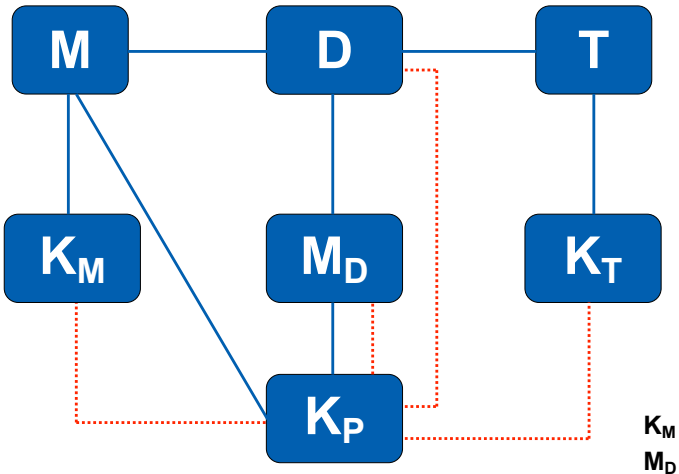





# Detaliem $P_C$



# Detaliem M & T




Flux Date

Flux Control/Comenzi Stare

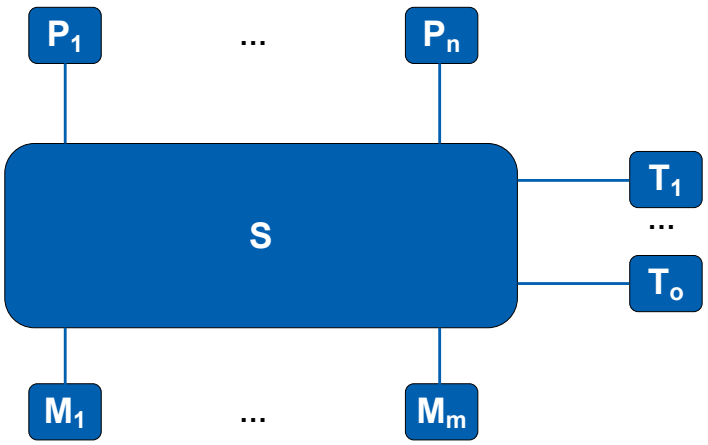
$K_M$ – Unitate de comanda a memoriei

$M_D$ – set de registrii generali

$K_P$ – procesorul ce interpreteaza si executa

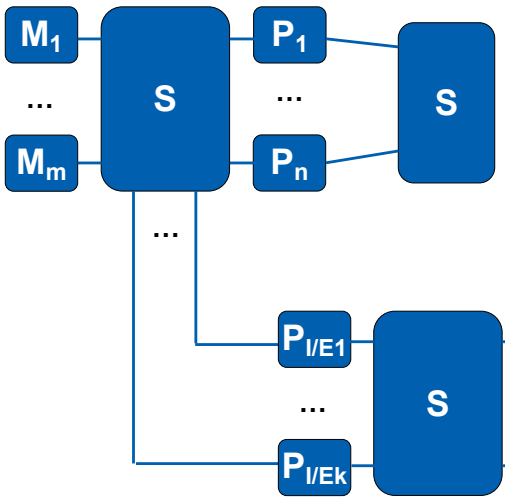


# Sistem cu mai multe Procesoare si Memorii



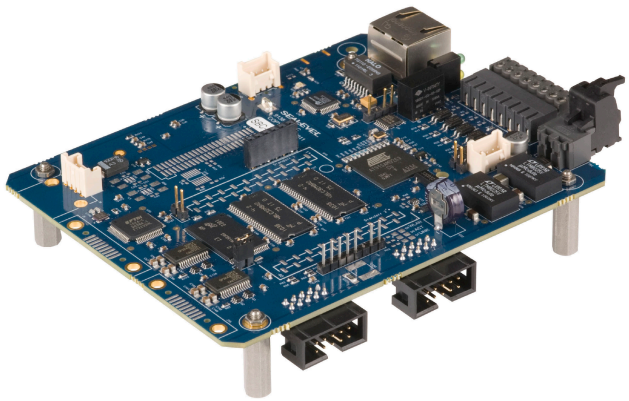

**S este cel mai important element pentru ca asigura conexiunile intre $P_i \leftrightarrow M_j$ sau $P_k \leftrightarrow T_l$**

# Sisteme Multiprocesor




- Memoriile sunt partajate de procesoare
- $P_{I/E}$ pot conecta orice T la orice P

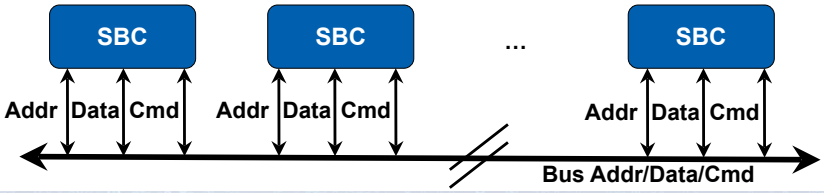




# Single Board Computers (SBC)

# Single Board Computer

- Are procesor, memorie + interfete I/O
- Conecteaza pe o magistrala comenzi, date si adrese bidirectional
- Resurse SBC:
  - Private: procesor, EPROM, Interfete (S/P), RT Clock, Sistem de Intreruperi
  - Locale: Memoria locala
  - Globale: RAM (in spatiul de adresare)

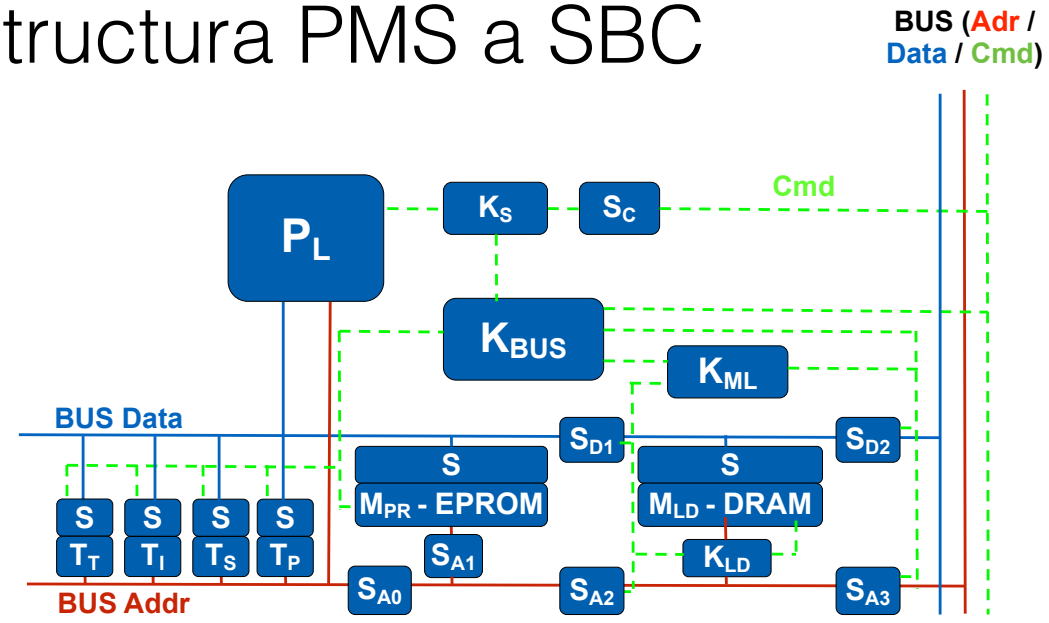





# SBC – Mecanisme Acces Magistrala

- Pentru a gestiona accesul la magistrala (exclusiv) sunt necesare:
  - $K_{BUS}$: unitate de comanda pt accesul la MAG
  - $K_{ML}$: unitate de control al accesului la memoria locala
  - $K_{LD}$: unitate de comanda a memoriei locale (refresh/etc)
  - $K_S$: unitate de comanda a intregului sistem – asigura controlul procesoarelor la resursele sistemului
- Fiecare procesor al SBC-ului vede memoria locala incapand de la 0 si memoriile celorlalte SBC-uri in spatii adiacente superioare

# Structura PMS a SBC






# Elementele Structurii PMS a SBC

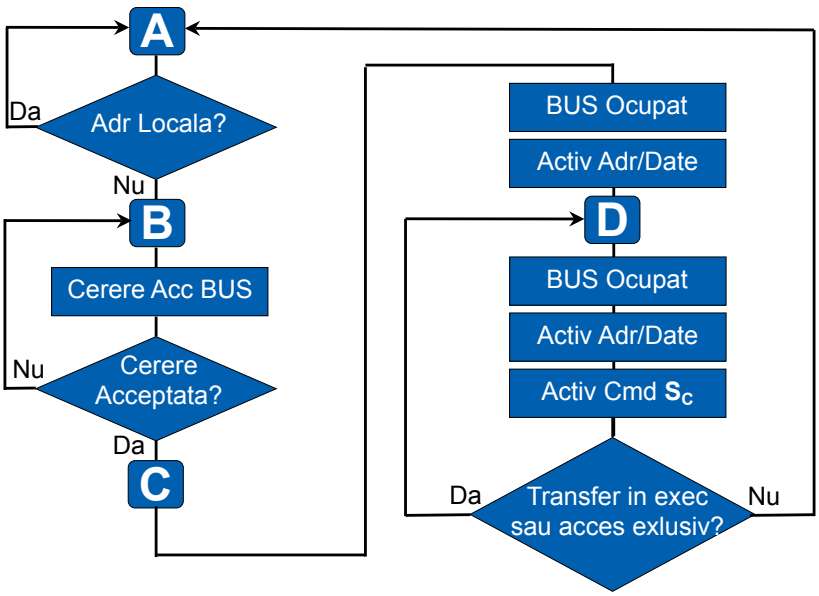
- $P_L$: procesor local – furnizeaza date si adrese
- Interfete:
  - $T_T$ – de ceas de timp real
  - $T_I$ – sistemului de intreruperi
  - $T_S$ – seriala & $T_P$ – paralela
  - Fiecare interfata are adresele portului sau de I/O
  - Daca isi recunoaste adresa isi pune datele pe magistrala
- $M_{LD}$ – DRAM: memoria poate fi accesata de $P_L$ si de procesoare din exterior
  - Exista niste S-uri care acorda accesul fie procesorului local fie celor externe
- $S_{D1}$ si $S_{D2}$ S-uri locale de date – separa procesoarele intre ele
- $S_{A0}$ si $S_{A1}$ – S-uri interne de adrese
- $S_{A2}$ si $S_{A3}$ – S-uri externe de adrese
- $S_C$ – S prin care se transmit comenzi catre exterior
- Obs: structura PMS poate fi detaliata pe componente cu directive PMS

# SBC – Mecanisme Acces Magistrala

- $K_s$ e specifica fiecarui procesor
  - In procesoarele moderne e inclusa in chip
  - $K_s$ genereaza semnalele de control ale resurselor locale
  - Generarea se face prin interpretarea semnalelor de stare date de microprocesor
- Trebuie sa existe o unitate de comanda $K_{BUS}$:
  - Fiecare procesor trebuie sa intre intr-o stare de asteptare pana primeste dreptul de acces la magistrala
  - Procesorul face doar un ciclu care apoi se poate prelungi prin stari de $T_{WAIT}$
- $K_{ML}$ controleaza S-urile de adrese si de date
  - Accesul local sau extern la memorie
  - Daca deschide toate S-urile se ajunge la accesul direct al procesorului local in exterior



# $K_{BUS}$



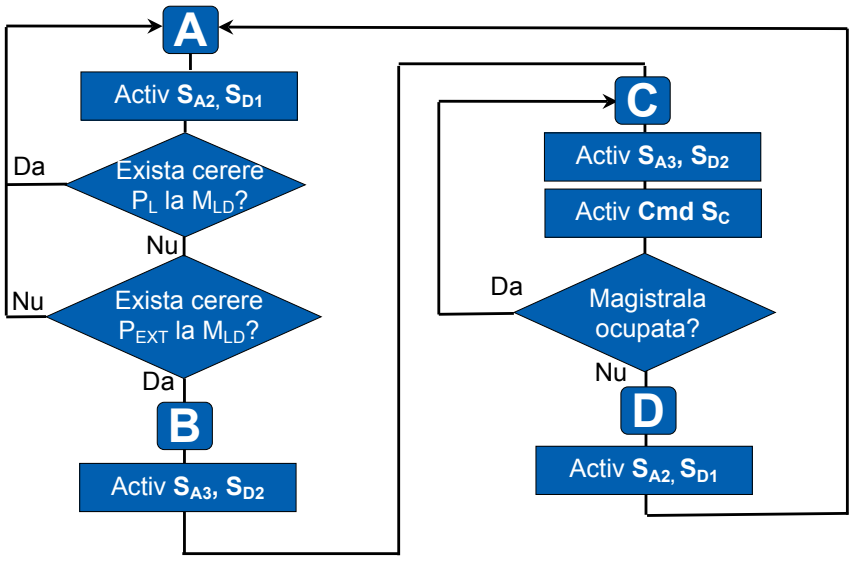
# Descriere $K_{BUS}$

- $K_{BUS}$ – reprezinta procesorul local in relatie cu magistrala externa
- Starea A: adresa e locala?
  - Da = se adreseaza resurse locale, nu trebuie iesit pe magistrala
  - Nu = adr nu e locala → se trece in starea B
- Starea B: cerere de acces la magistrala; daca cererea e acceptata se trece in starea C
- Starea C: se emite Bus Busy (mag ocupata) – doar in C & D; accesul pe BUS e exclusiv & cu rafala (overwrite) raman in D pt un nou ciclu daca mai e cazul; automatul preia functia procesorului; Dupa preluarea comenzii, procesorul face functia de secventiere a Adr, Date, Cmd
- Intrebare: de ce si starea C si starea D? Din cauza ciclului instructiune curent. Trebuie sincronizate comenzile cu decodificarea adreselor & stabilirea datelor pe magistrala (e un $\Delta_t$ acolo = $T_{Wait}$ datorat interpretarii adreselor) → se introduce starea D pentru a compensa acest timp de latenta: starea de overwrite
- Obs: $K_{BUS}$ trebuie sa dispuna de unitati logice combinationale ce identifica:
  - Daca adresa mem/port I/O implicate in transfer sunt locale
  - Semnalul de cerere acceptata e combinat din mai multe conditii: mag libera & modul SBC activ





# $K_{ML}$



# Descriere $K_{ML}$

- $K_{ML}$ – asigura accesul la Mem locala a PL prin intermediul $S_{D1}$ si $S_{A2}$ & a proc externe prin $S_{D2}$ si $S_{A3}$
- PL are prioritate fata de proc externe ($P_{EXT}$) == daca vrea $P_L$ la $M_{LD}$ NU ma intereseaza ce vor celelalte $P_{EXT}$
- Starea A: $P_L$ poate avea comenzi locale
- Starea B: activez accesul ext la $M_{LD}$ – semnalele asociate starii
- Obs: Mem locala e vazuta de fiecare procesor acelasi spatiu de memorie, iar de procesoarele ext in spatiul complementar → trebuie sa existe o unitate de translatare a adreselor
- Mag ocupata: nu stii ciclii masina ai unui proc extern. Numai cand magistrala e libera un $P_{EXT}$ a terminat ciclul masina curent → abordare top-down





# Ce am invatat in aces curs?

- Care sunt componentele sistemelor de calcul
- Processor Memory Switches - PMS
- Exemple de descrieri PMS
- Single Board Computers - Unitati de command





# In cursul urmator

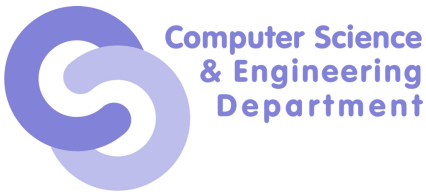
- Ierarhia de memorii – Bottleneck-ul SC
- Localitatea datelor
- Cache – design, implementari si exemple
- Imbunatatirea performantelor prin exploatarea ierarhiei de memorie





Thank you for your attention.




emil.slusanschi@cs.pub.ro
cs.pub.ro